

Assignment 4

R for Data Visualization: Fundamentals & First EDA

Team Number	
Project Title	
Dataset (name & source/link)	
Team Members	
Date	

Overview

This is your first hands-on session with R. It has three parts: getting R, RStudio, and the course packages installed and working; a short concept check drawing on the ideas from the Informational deck, so the vocabulary is solid before you start writing code; and a first R-based exploratory pass over your own project dataset — the same one you already explored in Python.

Part C mirrors the structure of your Python EDA (Assignment 2), just written in R. Comparing the two is part of the point: same dataset, same questions, a different language.

TIME ALLOTTED & WHAT TO SUBMIT

Part A — Installation & Setup: 45 minutes

Part B — Concept Check: 30 minutes

Part C — First EDA in R: 90 minutes

What to submit: installation screenshots, your concept-check answers, and labeled R plots with interpretation

Learning Objectives

- Install and configure R, RStudio, and the packages this course uses (ggplot2, dplyr, data.table, tidyr, reshape2, lubridate).
- Demonstrate working knowledge of the statistics, data manipulation, and visualization concepts from the Informational deck before applying them in code.
- Load your own project dataset into R and reproduce an early exploratory pass, building on the instincts you practiced in Python.

Rules

- Use your team's project dataset — the same file you used for Assignment 2, now loaded into R.
- Part A can be completed individually before class if possible; Parts B and C should be completed together as a team.
- Use at least 2 of the 3 R graphics systems covered in class (Base R, Lattice, ggplot2)
- Every plot needs a caption/figure number and 1–3 sentences of interpretation directly underneath it.**
- Submit code and written answers together as one artifact (see Deliverables). The file name should be your *<Registration Number>_<Team Number>_ Assignment4.docx*.

The Worksheet

Complete Parts A, B, and C in order. Part A gets your environment running, Part B checks your understanding of the concepts from class, and Part C puts both to work on your own data.

Part A: Installing R, RStudio & Your Packages

A.1 Install R and RStudio.

1. Go to cran.r-project.org and download R for your operating system (Windows / Mac / Linux). Run the installer and accept the default settings.
2. Go to posit.co/download/rstudio-desktop and download RStudio Desktop (free). Install it — RStudio will automatically detect your R installation. R must be installed before RStudio.
3. Open RStudio and confirm you can see the Console, Environment, and Files panes.

A.2 Install the packages this course uses.

Open the Console in RStudio and run:

```
install.packages(c('tidyverse', 'data.table', 'lubridate', 'reshape2'))

library(tidyverse)    # bundles ggplot2, dplyr, tidyr, readr, and more
library(data.table)
library(lubridate)
library(reshape2)
```

install.packages() downloads from CRAN — run once per machine. library() loads a package for the current session — run every time you restart R.

A.3 Load your own project dataset.

Unlike a built-in dataset, your project data needs to be read in from a file. Place your dataset in your working directory (or note its full file path), then:

```
# CSV file
df <- read.csv('your_dataset.csv')

# Excel file (needs the readxl package)
# install.packages('readxl')
# df <- readxl::read_excel('your_dataset.xlsx')

# Quick checks
dim(df)      # rows x columns
names(df)    # column names
str(df)      # data type of each column
head(df)     # first 6 rows
```

Tip: getwd() shows your current working directory; setwd('path/to/folder') changes it. In RStudio, Session > Set Working Directory > To Source File Location is usually the easiest route.

TASK A — INSTALLATION & DATA LOADING

Paste screenshots or console output for each of the following:

- `install.packages()` and `library()` running without errors for all four packages.
- `dim(df)`, `names(df)`, and `str(df)` for your own dataset.
- `head(df)` showing the first 6 rows of your data.

Part B: Concept Check — From the Informational Deck

Answer each question in your own words, in 1–3 sentences. No code needed here — this section checks that the vocabulary from class is solid before you start writing R.

B.1 In your own words, distinguish descriptive statistics from inferential statistics. Give one example of each using your own project dataset.

B.2 Suppose your key numeric variable has a few extreme outliers. Which measure of central tendency would you trust more — the mean or the median — and why? Name one measure of spread that is similarly robust to outliers.

B.3 What two parameters define a normal distribution, and what does each one control?

B.4 Name one sampling method other than simple random sampling, and describe a situation from your project domain where it would be the better choice.

B.5 Explain, in a sentence or two, what a p-value below your chosen significance level (α) tells you about the null hypothesis.

B.6 What is the difference between base R, the tidyverse, and `data.table` as approaches to working with data? Name one advantage of each.

B.7 Which single dplyr verb would you use to (a) keep only the rows meeting a condition, and (b) collapse many rows into one summary row per group?

B.8 Explain the difference between wide and long data format. Name one function — from `reshape2` or `tidyr` — that converts data from wide to long.

B.9 List the seven layers of `ggplot2`'s Grammar of Graphics, in order.

B.10 Your scatter plot has hundreds of points overlapping in one corner. Name two techniques you could use in `ggplot2` to make the underlying pattern more visible.

Part C: First EDA in R

This mirrors your Python EDA (Assignment 2) — same dataset, same kinds of questions, now answered in R. Each question expects at least one figure (unless noted) plus written interpretation.

Part C0: Setup & Data Overview

C0.1 Load your dataset in R and report its shape.

Include: number of rows, number of columns, column names and their types (`str()` or `summary(df, class)`). One or two sentences: does this match what you expected when you chose the dataset?

C0.2 Compute summary statistics for your numeric columns (`summary(df)`).

Which variable has the widest range? Which has the largest gap between mean and median — what might that suggest about its distribution?

C0.3 Quantify missing data by column (`colSums(is.na(df))`).

A table or a simple bar chart of missing-value counts per column is enough. Note any column you may need to drop or impute later.

Part C1: Univariate Exploration

C1.1 Plot the distribution of the single numeric variable most central to your project's research question.

Use `hist()` or `ggplot2`'s `geom_histogram()`. Is it symmetric, skewed, multi-modal? Does the shape suggest anything about the underlying process generating this data?

C1.2 Plot a bar chart of category counts for one key categorical variable.

`barplot(table(...))` or `ggplot2`'s `geom_bar()`. Are the categories roughly balanced or heavily skewed toward a few values?

C1.3 (Optional / bonus) Use a box plot to check your key numeric variable for outliers.

`boxplot()` or `geom_boxplot()`. If you find outliers, do they look like data errors, or genuinely interesting extreme cases?

Part C2: Relationships Between Variables

C2.1 Choose two numeric variables you think might be related and create a scatter plot.

`plot()` or `ggplot2`'s `geom_point()`. Describe the relationship: positive/negative/none, linear or not, strong or weak.

C2.2 If you have 3+ numeric variables, compute a correlation matrix (`cor()`) and visualize it as a heatmap.

A simple `ggplot2` `geom_tile()` or `Lattice`'s `levelplot()` both work. Which pair is most strongly correlated? Is there a correlation that surprised you — or one you expected that isn't there?

C2.3 Recreate one plot above using a second R graphics system.

If you used `ggplot2` above, try the same relationship in `Lattice` (`xyplot()`) or Base R (`plot()`). What did switching systems change about how easy the plot was to make, or to read?

Part C3: Group & Categorical Comparisons

C3.1 Group your data by a key categorical variable and compare a numeric outcome across groups.

`aggregate()`, or `dplyr`'s `group_by()` + `summarise()`, or a boxplot by group. What differences stand out between groups?

C3.2 If your dataset has a time/sequence dimension, plot your key variable over time. Otherwise, compare a second categorical grouping.

Any trend, seasonality, or turning point worth flagging?

Part C4: Data Quality Notes

C4.1 List up to 3 data quality issues you noticed while making today's plots.

For each issue (e.g. outliers, missing-value clustering, inconsistent category labels, unit mismatches), name which chart revealed it to you.

Part C5: Synthesis & Reflection

C5.1 Write 3 key findings from today's R EDA, 2–3 sentences each, each referencing a specific figure.

E.g. "Figure 3 shows that ... which suggests ..." Avoid restating the chart — say what it means.

C5.2 Which R graphics system — Base R, Lattice, or ggplot2 — felt most natural for this dataset, and why?

C5.3 Compare today's R findings to your original Python EDA from Assignment 2. Did anything look different, or reveal something new?

C5.4 What is one question about your dataset you still want to answer?

Deliverables

- A document file with all written answers — Task A screenshots, Part B concept-check answers, and Part C's labeled/captioned plots.
- Your .R script (or .Rmd file) containing all code from Parts A and C.
- Every figure numbered (Figure 1, Figure 2, ...) and referenced by number in your written answers.
- File name: *<Registration Number>_<Team Number>_Assignment4.docx*